Artificial Intelligence Lab Manual

Practical Implementation of Local Search and Optimization

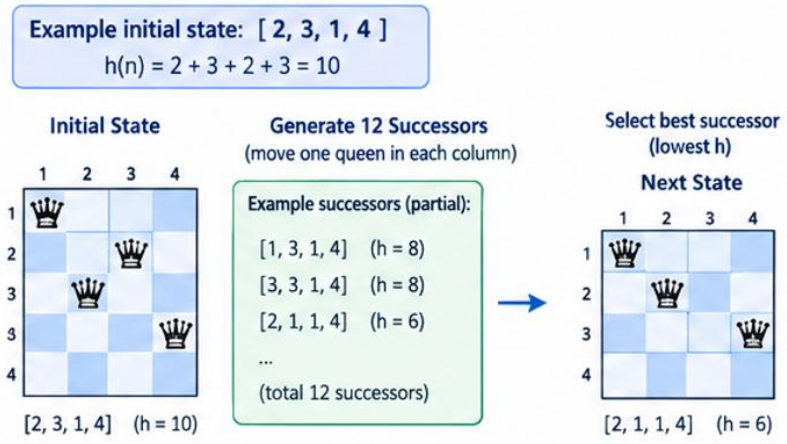
CLO - 2: Apply classical AI techniques, including uninformed and informed search algorithms, minimax, and neural networks, to solve structured problems

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the.ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**

Task 1: Implement Hill Climbing for N-Queens

Problem Scenario: Implement a local-search solver for the 4-Queens problem. A state represents one queen in each column of a 4×4 board. A queen may be moved to another row within the same column. Therefore, each state has 4×3 = 12 possible successor states, matching the lecture example. The objective is to minimize h(n), where h(n) represents the number of queen conflicts involving the same row, column, or diagonal. A solution is reached when h(n)=0.



Requirements

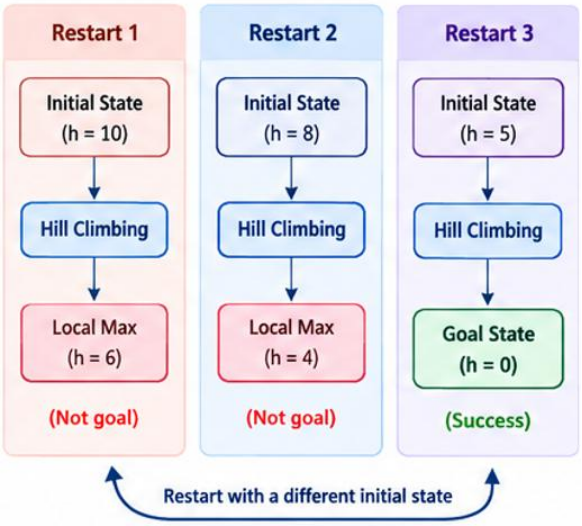
- Represent a board as a list of four integers, where index = column and value = row.
- Generate every legal neighbor by moving exactly one queen to another row in the same column.
- Implement a conflict-counting function h(state).
- Start from a randomly generated state.
- Evaluate all 12 neighbors and move to the neighbor with the lowest h-value.
- Continue until h=0 or no neighbor improves the current state.
- Print the state and h-value at every iteration.
- Display the final board using Q for a queen and - for an empty cell.

Expected Output

A trace similar to: Initial State → h(initial) → Best Neighbor → h(best neighbor) → ... → Solution (h=0), or a clear message indicating that the algorithm became stuck.

Task 2: Random-Restart Hill Climbing

Extend Task 1 so that the algorithm does not rely on one initial state. In the theory lecture's slides it has already been discussed that random-restart hill climbing as a solution to the local-optimum problem: when one search fails, the algorithm starts again from a different initial state and repeats until a solution is obtained.



Requirements

- Reuse your hill-climbing implementation from Task 1.
- Run hill climbing from a new randomly generated initial state whenever the previous run fails.
- Stop when a state with h=0 is found or when a maximum number of restarts is reached.
- Record the number of restarts, iterations per restart, and final solution.
- Run the experiment at least 10 times and calculate the average number of restarts required.
- Compare the success behavior of one-start hill climbing and random-restart hill climbing.

Suggested Experimental Table

Run	Initial State	Restarts	Total Iterations	Final h

Task 3: Local Beam Search for N-Queens

Implement Local Beam Search for the same N-Queens representation. Instead of following one state at a time, maintain k states simultaneously. Begin with k randomly generated states, generate all successors

of those states, and select the best k successors according to the heuristic. Repeat until h=0 or a maximum number of iterations is reached.

Requirements

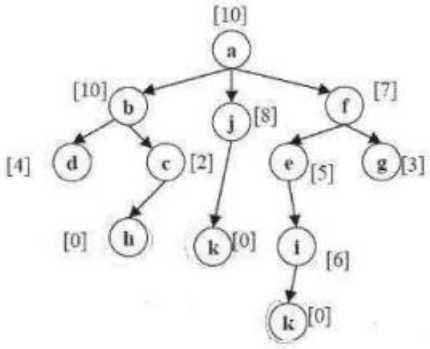
- Allow the user to enter the beam width k.
- Generate k initial random states.
- Generate all successors for every retained state.
- Combine the successors into one candidate pool.
- Sort/select the k states with the lowest h-values.
- Remove duplicate states before or during selection so that the beam contains useful diversity.
- Repeat the process until a solution is found or the iteration limit is reached.
- Run the program for k=1, k=2, k=3, and k=5 and record the results.

Task 4: Beam Search on a Heuristic Graph

Implement Beam Search on the given heuristic graph to find the goal node K. Use a beam width of k = 2. Starting from node a, generate the children of the current beam nodes, evaluate the heuristic values of the generated nodes, and retain only the best two candidates for the next level. Continue the process until node K is selected as the goal. Display the current beam, generated children, heuristic values, and selected best k successors at each level.

Graph Specification

Create the graph from the node relationships provided by the lecture's Beam Search example. Assign each node the heuristic value shown in the lecture diagram. If a value is not legible in the supplied PDF, do not invent it; use the clearly readable values or ask the instructor for the original diagram.



Requirements

- Represent the graph using an adjacency-list data structure.
- Store each node's heuristic value separately.
- Initialize the beam with the specified starting node(s).
- Generate children of all nodes currently in the beam.
- Rank the generated successors by heuristic value.
- Retain only the best k candidates.
- Repeat until K is selected as the goal or the search terminates without finding K.
- Print the beam at every iteration so the selection process is visible.

Required Comparison

Run the implementation with k=2 and at least one other value of k. Report how the selected branches change and how the beam width affects the number of alternatives explored simultaneously.

Task 5: Simulated Annealing for N-Queens

Implement the Simulated Annealing algorithm to solve the 4-Queens problem. Represent each state as a configuration containing one queen in each column of the board, and use a heuristic function h(n) representing the number of queen conflicts. Unlike hill climbing, the algorithm should sometimes accept a worse neighboring state to escape local optima. Start with a high temperature and gradually reduce it according to a cooling schedule. The algorithm should terminate when a solution with h(n) = 0 is found or the temperature falls below a specified threshold. Display the current state, heuristic value, temperature, and whether each generated neighbor was accepted or rejected.

Requirements

1. Generate a random initial 4-Queens state.
2. Generate a random neighboring state by moving one queen within its column.
3. Calculate the change in heuristic value between the current and neighboring states.
4. Always accept a better state.
5. Sometimes accept a worse state according to the simulated annealing acceptance probability.
6. Gradually decrease the temperature using a cooling schedule.
7. Stop when h(n) = 0 or the temperature becomes sufficiently small.
8. Display the search progress during execution.
9. Experiment with at least two different cooling rates and compare the results.

Implementation Guidance

Keep the implementation modular. A recommended structure is:

- generate_initial_state()
- evaluate_state(state)
- generate_neighbors(state)
- hill_climb(initial_state)
- random_restart_hill_climb(max_restarts)
- local_beam_search(k, max_iterations)
- beam_search(graph, heuristic, start, goal, k)

Keep the state representation independent from the search algorithm. This makes it possible to reuse the same N-Queens representation across multiple algorithms and makes the comparison more meaningful.